

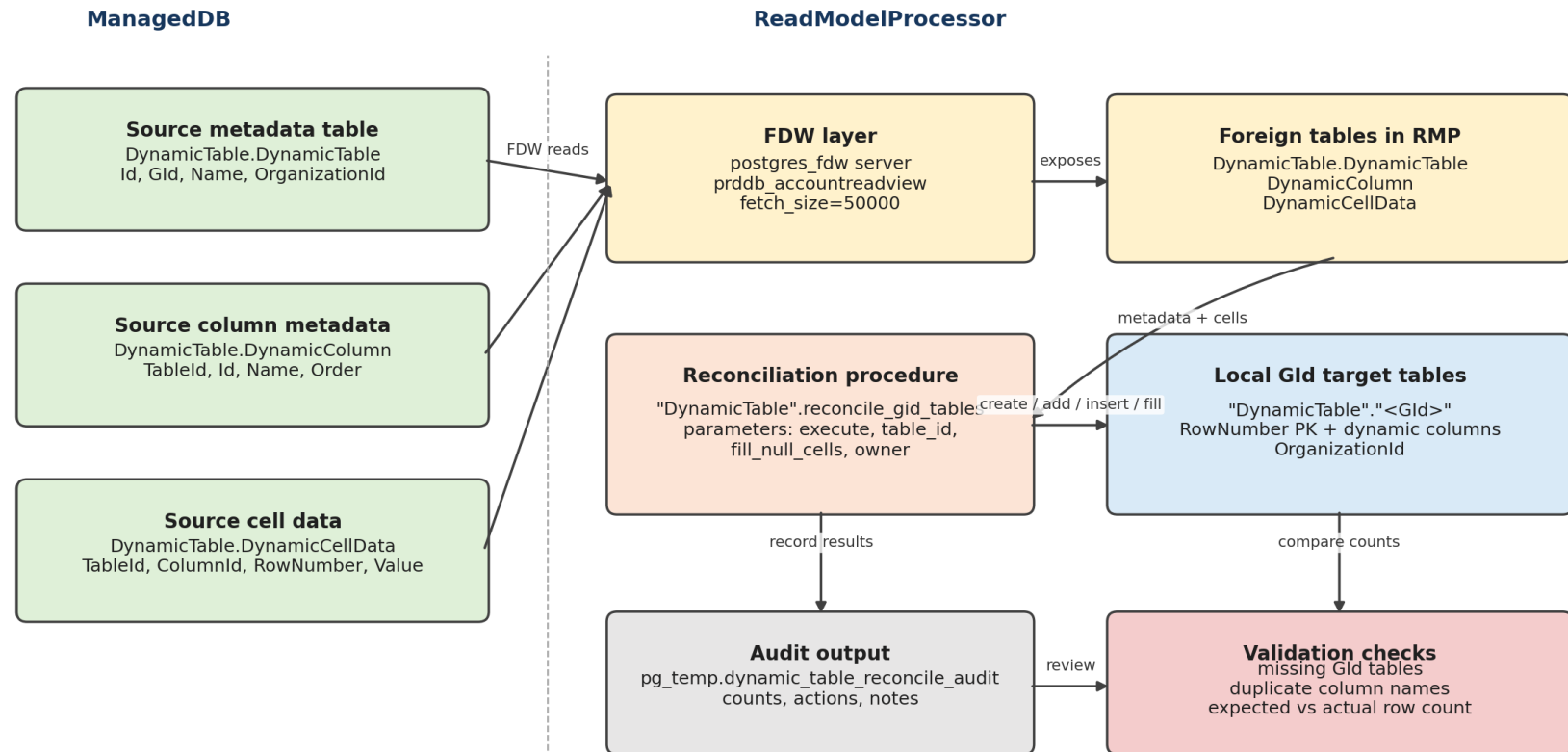
DynamicTable Data Sync Reconciliation

Technical document and data flow block diagram for repairing local GId tables in ReadModelProcessor from ManagedDB foreign tables.

Area	Confirmed design
Source database	ManagedDB
Read database	ReadModelProcessor
FDW server	prddb_accountreadview using postgres_fdw
Source objects in RMP	Foreign tables: DynamicTable, DynamicColumn, DynamicCellData
Target objects in RMP	Local physical tables named by DynamicTable.GId under schema "DynamicTable"
Repair strategy	Create missing GId tables, add missing columns, insert missing rows, optionally fill NULL cells

1. Data flow block diagram

The diagram below shows the confirmed direction of data movement and the separation between ManagedDB source data, FDW exposure, and ReadModelProcessor local GId target tables.



Data repair direction: ManagedDB source foreign tables -> ReadModelProcessor local physical GId tables. The process does not delete extra target rows.

Key rule: the repair operation flows from the foreign table data into local GId tables. It should not delete rows from the local target tables unless a separate, explicitly approved cleanup script is created.

2. System context

ReadModelProcessor contains the schema "DynamicTable". Inside that schema, the three metadata and cell source objects are foreign tables exposed through postgres_fdw, while the user-facing dynamic data tables are ordinary local tables named after DynamicTable.GId.

The screenshots and DDL confirm that both the foreign tables and local GId tables are visible under the same schema. Therefore, the reconciliation logic must distinguish foreign source tables from ordinary local target tables.

Object	Type	Purpose
"DynamicTable"."DynamicTable"	Foreign table	Lists dynamic tables. Important fields include Id, GId, Name, OrganizationId, StatusId.
"DynamicTable"."DynamicColumn"	Foreign table	Lists dynamic columns per TableId. Column Name is used as the local physical column name.
"DynamicTable"."DynamicCellData"	Foreign table	Stores cell values by TableId, ColumnId, RowNumber, and Value.
"DynamicTable"."<GId>"	Local table	Physical read model table. Contains RowNumber primary key, dynamic varchar columns, and OrganizationId.

3. Foreign server and foreign table design

The foreign server prddb_accountreadview points from ReadModelProcessor to ManagedDB. The three foreign tables are not copied locally by this DDL; they are pointers used to read ManagedDB data through postgres_fdw.

```
CREATE SERVER prddb_accountreadview
  FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (
    host 'dev001dbgre20.postgres.database.azure.com',
    dbname 'ManagedDB',
    port '6432',
    fetch_size '50000',
    async_capable 'true'
  );

CREATE FOREIGN TABLE IF NOT EXISTS "DynamicTable"."DynamicCellData"(
  "Id" bigint NOT NULL,
  "TableId" bigint NOT NULL,
  "ColumnId" bigint NOT NULL,
  "RowNumber" bigint NOT NULL,
  "Value" character varying(10000),
  "ColorGroupId" bigint
)
SERVER prddb_accountreadview
OPTIONS (schema_name 'DynamicTable', table_name 'DynamicCellData');
```

4. Local target table pattern

Each dynamic table is materialized as a local physical table in ReadModelProcessor. The table name is the GId value from the foreign DynamicTable metadata row.

```
CREATE TABLE IF NOT EXISTS "DynamicTable"."0a04b818-91da-40f6-a022-3a1cbb6e8a42" (  
  "RowNumber" bigint NOT NULL,  
  "Date" character varying,  
  "Price" character varying,  
  "Open" character varying,  
  "High" character varying,  
  "Low" character varying,  
  "Vol." character varying,  
  "Change %" character varying,  
  "Created By" character varying,  
  "Updated By" character varying,  
  "Created Date" character varying,  
  "Updated Date" character varying,  
  "OrganizationId" bigint,  
  CONSTRAINT "0a04b818-91da-40f6-a022-3a1cbb6e8a42_pkey" PRIMARY KEY ("RowNumber")  
);
```

5. Reconciliation procedure behavior

The procedure "DynamicTable".reconcile_gid_tables processes either one table or all tables. The p_table_id parameter is optional. Passing NULL means all DynamicTable rows with a GId are selected; passing a specific Id restricts the run to one table.

```
FOR r_table IN
  SELECT dt."Id" AS table_id,
         dt."GId"::uuid AS gid,
         dt."OrganizationId" AS organization_id,
         dt."StatusId" AS status_id,
         dt."Name" AS table_name
  FROM "DynamicTable"."DynamicTable" dt
  WHERE dt."GId" IS NOT NULL
        AND (p_table_id IS NULL OR dt."Id" = p_table_id)
  ORDER BY dt."Id"
LOOP
  -- r_table.table_id drives DynamicColumn and DynamicCellData filters
END LOOP;
```

Step	Action	Result
1	Read DynamicTable metadata	Get table Id, GId, OrganizationId, StatusId, and Name.
2	Check target object in pg_class	Only relkind r or p is accepted as a local target table. Foreign tables are not treated as repair targets.
3	Count expected rows	Count distinct DynamicCellData.RowNumber for the current TableId.
4	Detect duplicate columns	Skip if DynamicColumn.Name duplicates would create invalid duplicate target columns.
5	Create missing target table	Build RowNumber PK, dynamic varchar columns from DynamicColumn.Name, and OrganizationId.
6	Add missing columns	ALTER TABLE ADD COLUMN IF NOT EXISTS for all DynamicColumn.Name values plus OrganizationId.
7	Pivot cell data	Aggregate DynamicCellData by RowNumber and ColumnId into one row per target RowNumber.
8	Insert missing rows	Insert only rows that do not already exist in the GId table by RowNumber.
9	Fill NULL cells	When enabled, fill target NULL values from source cell data without overwriting existing non-NULL values.
10	Write audit row	Record expected rows, existing rows, inserted rows, updated rows, duplicate column count, and note.

6. Procedure parameters

Parameter	Default	Meaning
p_execute boolean	false	false performs a dry run. true performs DDL/DML changes.
p_table_id bigint	NULL	NULL processes all tables. A value processes only that DynamicTable.Id.
p_fill_null_cells boolean	true	When true, fills NULL target cells from source values. Existing non-NULL values are not overwritten.

Parameter	Default	Meaning
p_owner name	NULL	Optional owner to assign to newly created local Gld tables, for example lobdev.

7. Recommended execution workflow

Run the procedure first in a transaction and roll it back after reviewing the audit table. After validating the audit output in staging or a restored backup, run the procedure with p_execute = true.

Dry run for all tables

```
BEGIN;

CALL "DynamicTable".reconcile_gid_tables(false, NULL, true, 'lobdev');

SELECT *
FROM pg_temp.dynamic_table_reconcile_audit
ORDER BY table_id;

ROLLBACK;
```

Execute for all tables

```
BEGIN;

CALL "DynamicTable".reconcile_gid_tables(true, NULL, true, 'lobdev');

SELECT *
FROM pg_temp.dynamic_table_reconcile_audit
ORDER BY table_id;

COMMIT;
```

Execute for one table only

```
BEGIN;

CALL "DynamicTable".reconcile_gid_tables(true, 832, true, 'lobdev');

SELECT *
FROM pg_temp.dynamic_table_reconcile_audit
ORDER BY table_id;

COMMIT;
```

8. Validation and safety notes

Topic	Guidance
Backups	Test on staging or a restored backup before production. The procedure can create tables and insert data across approximately 2,000 tables.
No deletes	The proposed script does not delete extra target rows. This is intentional because sample data showed the GId table may have more rows than DynamicCellData.

Topic	Guidance
No overwrite by default	Existing non-NULL target values are preserved. Only missing rows and NULL target cells are repaired.
Duplicate column names	Tables with duplicate DynamicColumn.Name values are skipped and reported in the audit output.
Object type check	The target existence check accepts only ordinary or partitioned local tables, not foreign tables.
Performance	Run in batches if needed by passing p_table_id for high-volume tables, and monitor FDW query load on ManagedDB.
Locking	CREATE TABLE and ALTER TABLE take DDL locks. Schedule during a low-traffic window for production.

9. Open confirmations

The current script assumes DynamicColumn.Name is the physical target column name and that all dynamic columns should be character varying, matching the provided GId table DDL. If a separate normalized/system column field should be used instead, the CREATE TABLE, ALTER TABLE, INSERT, and UPDATE generation logic should be adjusted before production use.

The current logic also assumes OrganizationId should be populated from DynamicTable.OrganizationId on inserted rows and filled when the target OrganizationId is NULL.

10. Appendix - main pre-check queries

```
-- GId values that do not have a local ordinary or partitioned table.
SELECT dt."Id" AS table_id,
       dt."Name" AS table_name,
       dt."GId",
       dt."OrganizationId",
       c.relkind AS existing_object_relkind
FROM "DynamicTable"."DynamicTable" dt
LEFT JOIN pg_class c ON c.relname = dt."GId"::text
LEFT JOIN pg_namespace n ON n.oid = c.relnamespace AND n.nspname = 'DynamicTable'
WHERE dt."GId" IS NOT NULL
      AND NOT EXISTS (
        SELECT 1
        FROM pg_class c2
        JOIN pg_namespace n2 ON n2.oid = c2.relnamespace
        WHERE n2.nspname = 'DynamicTable'
              AND c2.relname = dt."GId"::text
              AND c2.relkind IN ('r', 'p')
      )
ORDER BY dt."Id";

-- Duplicate target column names. These must be resolved before repair.
SELECT dc."TableId", dc."Name", count(*) AS duplicate_count
FROM "DynamicTable"."DynamicColumn" dc
GROUP BY dc."TableId", dc."Name"
HAVING count(*) > 1
ORDER BY dc."TableId", dc."Name";
```



```
-- Source cell count by table.
SELECT dt."Id" AS table_id,
       dt."GId",
       count(DISTINCT dcd."RowNumber") AS expected_rows_from_cell_data,
       count(*) AS source_cell_count
FROM "DynamicTable"."DynamicTable" dt
LEFT JOIN "DynamicTable"."DynamicCellData" dcd ON dcd."TableId" = dt."Id"
WHERE dt."GId" IS NOT NULL
GROUP BY dt."Id", dt."GId"
ORDER BY dt."Id";
```